

Informe Técnico Final: Sistema de Información con CRUD e Integración de IA Local (Ollama)

Asignatura: Programación IV — Actividad 5

Portada del Proyecto

- **Título del Proyecto:** Sistema de Gestión de Inventario Tecnológico con Trazabilidad Kardex, Reportes Gerenciales y Asistente Analítico Offline mediante Ollama
 - **Estudiante:** Gabriel Alcón
 - **Asignatura:** Programación IV
 - **Docente:** Ing. Jared López Leaños
 - **Asistente de Codificación IA:** Google Antigravity (Advanced Agent IDE)
 - **Fecha de Entrega:** 28 de Septiembre de 2026
 - **Herramientas y Entorno de Ejecución:** Python 3.11.2, Django 5.1.6, Ollama 0.4.7+ (qwen2.5:1.5b), Google Antigravity, SQLite 3 (Modo WAL), Git, Debian GNU/Linux 12 (Bookworm)
-

Punto 1: Diseño e implementación del CRUD (30 pts)

1.1 Definición de la entidad y modelo de datos

Para satisfacer el escenario de gestión de información empresarial se seleccionó la entidad **Producto**, orientada al comercio y distribución de componentes tecnológicos y hardware de alta gama en Bolivia (moneda oficial: Bolivianos, Bs.), desarrollada sobre el framework Django (Django Software Foundation, 2024).

El modelo cumple holgadamente con el requisito mínimo de 6 campos (contando con 13 atributos en total), asegurando integridad referencial, indexación de alto desempeño y tipado estricto. En la Tabla 1 se

presenta la ficha técnica del modelo de datos:

Tabla 1

Especificación de Atributos, Tipos de Datos y Restricciones del Modelo Producto

Nombre del Campo	Tipo de Dato en Django	Modificadores / Constraints	Propósito Funcional
codigo	CharField(max_length=50)	unique=True , obligatorio	Identificador único empresarial (SKU de referencia).
nombre	CharField(max_length=200)	obligatorio	Denominación comercial completa del artículo.
descripcion	TextField	blank=True	Descripción detallada de uso y características.
categoria	CharField(max_length=100)	db_index=True , obligatorio	Agrupación taxonómica (ej. Tarjetas Gráficas, Procesadores).
precio	DecimalField(max_digits=10, decimal_places=2)	db_index=True , \$lte 0\$	Precio de venta en moneda nacional (Bolivianos, Bs.).
cantidad_existente	PositiveIntegerField	default=0 , \$lte 0\$	Existencias físicas reales en almacén.
stock_minimo	PositiveIntegerField	default=5 , \$lte 0\$	Umbral crítico para generación de alertas de reposición.
estado	BooleanField	default=True , db_index=True	Indicador de eliminación lógica (Activo / Inactivo).
fecha_registro	DateTimeField	auto_now_add=True	Estampa temporal inmutable de alta en el sistema.

Nombre del Campo	Tipo de Dato en Django	Modificadores / Constraints	Propósito Funcional
fecha_actualizacion	DateTimeField	auto_now=True	Registro automático del último cambio en el catálogo.
marca	CharField(max_length=100)	blank=True	Fabricante original del componente (ASUS, AMD, Corsair, etc.).
especificaciones	TextField	blank=True	Ficha técnica y parámetros de hardware.
destacado	BooleanField	default=False	Bandera para promoción en portada o catálogo primario.

Nota. Elaboración propia basada en la especificación del modelo de datos para persistencia en SQLite (Django Software Foundation, 2024).

Fragmento de Código: `chatbot/app/models.py`

```

from django.db import models

class Producto(models.Model):
    codigo = models.CharField(max_length=50, unique=True, verbose_name="Código del Producto")
    nombre = models.CharField(max_length=200, verbose_name="Nombre del Producto")
    descripcion = models.TextField(blank=True, verbose_name="Descripción")
    categoria = models.CharField(max_length=100, db_index=True, verbose_name="Categoría")
    precio = models.DecimalField(max_digits=10, decimal_places=2, db_index=True, verbose_name="Precio (Bs.)")
    cantidad_existente = models.PositiveIntegerField(default=0, verbose_name="Cantidad Existente")
    stock_minimo = models.PositiveIntegerField(default=5, verbose_name="Stock Mínimo")
    estado = models.BooleanField(default=True, db_index=True, verbose_name="Estado del Producto")
    fecha_registro = models.DateTimeField(auto_now_add=True, verbose_name="Fecha de Registro")
    fecha_actualizacion = models.DateTimeField(auto_now=True, verbose_name="Última Actualización")
    marca = models.CharField(max_length=100, blank=True, default="", verbose_name="Marca")
    especificaciones = models.TextField(blank=True, default="", verbose_name="Especificaciones Técnicas")
    destacado = models.BooleanField(default=False, verbose_name="Producto Destacado")

    class Meta:
        verbose_name = "Producto"
        verbose_name_plural = "Productos"
        ordering = ["-precio"]

    def __str__(self):
        estado_txt = "Activo" if self.estado else "Inactivo"
        return f"[{self.codigo}] {self.nombre} - Bs. {self.precio} ({estado_txt})"

    @property
    def estado_stock(self):
        if self.cantidad_existente == 0:
            return "Agotado"
        elif self.cantidad_existente <= self.stock_minimo:
            return "Stock Bajo"
        return "En Stock"

```

Complementariamente, para asegurar trazabilidad contable inmutable, se implementó el modelo `MovimientoStock` (Kardex):

```
class MovimientoStock(models.Model):
    TIPO_CHOICES = [
        ("ENTRADA", "Entrada (+)"),
        ("SALIDA", "Salida (-)"),
        ("AJUSTE", "Ajuste de Inventario"),
    ]
    producto = models.ForeignKey(Producto, on_delete=models.CASCADE, related_name="movimientos")
    tipo = models.CharField(max_length=10, choices=TIPO_CHOICES)
    cantidad = models.PositiveIntegerField()
    stock_previo = models.PositiveIntegerField()
    stock_resultante = models.PositiveIntegerField()
    motivo = models.CharField(max_length=255)
    usuario = models.CharField(max_length=100, default="Administrador")
    fecha = models.DateTimeField(auto_now_add=True)
```

1.2 Implementación en Django, migraciones y panel de administración personalizado

Comandos ejecutados y su salida de terminal

Para crear y aplicar el esquema relacional en SQLite:

```
python chatbot/manage.py makemigrations app
```

Salida obtenida:

```
Migrations for 'app':
  chatbot/app/migrations/0006_producto_fecha_actualizacion_and_more.py
    - Add field fecha_actualizacion to producto
    - Add field marca to producto
    - Add field especificaciones to producto
    - Add field destacado to producto
```

```
python chatbot/manage.py migrate
```

Salida obtenida:

```

Operations to perform:
  Apply all migrations: admin, app, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying app.0001_initial... OK
  Applying app.0002_alter_chathistory_options_and_more... OK
  Applying app.0003_producto... OK
  Applying app.0004_actualizar_producto... OK
  Applying app.0005_alter_producto_cantidad_existente_and_more... OK
  Applying app.0006_producto_fecha_actualizacion_and_more... OK
  Applying sessions.0001_initial... OK

```

```
python chatbot/manage.py runserver 0.0.0.0:8000
```

Salida obtenida:

```

Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
Django version 5.1.6, using settings 'chatbot.settings'
Starting development server at http://0.0.0.0:8000/
Quit the server with CONTROL-C.

```

Fragmento de Código: Panel de Administración Personalizado (`chatbot/app/admin.py`)

```

from django.contrib import admin
from .models import ChatHistory, Producto

@admin.register(Producto)
class ProductoAdmin(admin.ModelAdmin):
    list_display = ('codigo', 'nombre', 'categoria', 'marca', 'precio', 'cantidad_existente', 'estado', 'estado_stock', 'destacado')
    list_filter = ('estado', 'categoria', 'marca', 'destacado')
    search_fields = ('codigo', 'nombre', 'marca', 'especificaciones')
    list_editable = ('precio', 'cantidad_existente', 'estado', 'destacado')

```

1.3 Vistas, formularios y plantillas para el CRUD completo

El sistema cuenta con un flujo CRUD interactivo respaldado por formularios Django que realizan validación a nivel de servidor (unicidad, tipos de datos, valores no negativos) y controladores que retornan

retroalimentación con mensajes de éxito o error.

Fragmento de Código: Formularios con Validaciones (`chatbot/app/forms.py`)

```
from django import forms
from .models import Producto

class ProductoForm(forms.ModelForm):
    class Meta:
        model = Producto
        fields = ['codigo', 'nombre', 'descripcion', 'categoria', 'precio', 'cantidad_existente', 'stock_minimo', 'estado', 'marca', 'especificaciones']

    def clean_codigo(self):
        codigo = self.cleaned_data.get('codigo', '').strip()
        if not codigo:
            raise forms.ValidationError("El código del producto es obligatorio.")
        qs = Producto.objects.filter(codigo__iexact=codigo)
        if self.instance and self.instance.pk:
            qs = qs.exclude(pk=self.instance.pk)
        if qs.exists():
            raise forms.ValidationError(f"El código '{codigo}' ya está registrado con otro producto.")
        return codigo

    def clean_precio(self):
        precio = self.cleaned_data.get('precio')
        if precio is None or precio < 0:
            raise forms.ValidationError("El precio no puede ser negativo.")
        return precio

    def clean_cantidad_existente(self):
        cantidad = self.cleaned_data.get('cantidad_existente')
        if cantidad is None or cantidad < 0:
            raise forms.ValidationError("La cantidad existente no puede ser negativa.")
        return cantidad
```

Fragmento de Código: Controladores CRUD (chatbot/app/views.py)

```

@csrf_exempt
@require_http_methods(["POST"])
def api_crear_producto(request):
    """Creación de nuevo producto con validación y apertura en Kardex."""
    form = ProductoForm(request.POST)
    if form.is_valid():
        producto = form.save()
        if producto.cantidad_existente > 0:
            registrar_movimiento_kardex(
                producto=producto, tipo="ENTRADA", cantidad=producto.cantidad_existente,
                stock_previo=0, stock_resultante=producto.cantidad_existente,
                motivo="Registro inicial y alta en catálogo"
            )
        invalidar_cache()
        return JsonResponse({"status": "ok", "message": f"Producto '{producto.nombre}' regi
strado exitosamente."}, status=201)
        return JsonResponse({"status": "error", "errors": form.errors}, status=400)

@csrf_exempt
@require_http_methods(["POST"])
def api_editar_producto(request, pk):
    """Edición de producto con asiento de ajuste en Kardex."""
    producto = get_object_or_404(Producto, pk=pk)
    stock_anterior = producto.cantidad_existente
    form = ProductoForm(request.POST, instance=producto)
    if form.is_valid():
        prod_guardado = form.save()
        if stock_anterior != prod_guardado.cantidad_existente:
            delta = prod_guardado.cantidad_existente - stock_anterior
            registrar_movimiento_kardex(
                producto=prod_guardado, tipo="AJUSTE", cantidad=abs(delta),
                stock_previo=stock_anterior, stock_resultante=prod_guardado.cantidad_existe
nte,
                motivo=f"Ajuste manual de stock en edición ({delta:+d} uds)"
            )
        invalidar_cache()
        return JsonResponse({"status": "ok", "message": f"Producto '{prod_guardado.nombre}'
actualizado correctamente."})
        return JsonResponse({"status": "error", "errors": form.errors}, status=400)

@csrf_exempt
@require_http_methods(["POST", "DELETE"])
def api_eliminar_producto(request, pk):
    """Eliminación lógica (desactivación) para preservar integridad contable del Kardex."""
    producto = get_object_or_404(Producto, pk=pk)
    accion = request.POST.get("tipo Eliminacion", "logica").strip()
    if accion == "fisica":
        producto.delete()
        msg = "eliminado definitivamente del sistema."
    else:
        producto.estado = False
        producto.save()

```

```
msg = "desactivado lógicamente (Estado: Inactivo)."  
invalidar_cache()  
return JsonResponse({"status": "ok", "message": f"Producto '{producto.nombre}' {msg}"})
```

1.4 Implementación y resultados de los 8 reportes predefinidos

El sistema supera el requisito de 5 reportes, ofreciendo **8 reportes corporativos predefinidos** en `inventory_service.py` con síntesis de datos oficiales verificados:

```
curl -s http://127.0.0.1:8000/api/reportes/<tipo>/
```

Descripción textual de los resultados obtenidos en cada reporte:

1. Reporte 1: Catálogo Completo de Productos (`todos`)

2. *Descripción:* Muestra el inventario consolidado activo.

3. *Datos obtenidos:* Contiene exactamente **50 productos tecnológicos registrados**, todos en estado activo (`estado=True`), clasificados en sus 10 categorías oficiales.

4. Reporte 2: Producto Más Caro (`mas_caro`)

5. *Descripción:* Identifica el artículo de mayor precio unitario de venta.

6. *Datos obtenidos:* **ASUS ROG Strix GeForce RTX 4090 24GB OC Edition** (Código: `GPU-NV-4090-ROG`), perteneciente a la categoría *Tarjetas Gráficas*, con un precio unitario de **Bs. 26,839.88** y 8 unidades físicas disponibles.

7. Reporte 3: Producto Más Barato (`mas_barato`)

8. *Descripción:* Identifica el componente con el costo más accesible.

9. *Datos obtenidos:* **Crucial 8GB DDR4 3200MHz UDIMM** (Código: `RAM-CRU-8GB-D4`), en la categoría *Memorias RAM*, con un precio unitario de **Bs. 243.88** y 25 unidades en inventario.

10. Reporte 4: Productos con Pocas Existencias (`pocas_existencias`)

11. *Descripción:* Filtra artículos con existencia física `$le$` al stock mínimo establecido (sin llegar a cero).

12. *Datos obtenidos:* Se detectan **3 referencias críticas**:

- CPU-AMD-7800X3D (AMD Ryzen 7 7800X3D): 1 unidad disponible (stock mínimo fijado: 4 unidades).
- MB-ASU-Z790-HERO (ASUS ROG Maximus Z790 Dark Hero): 2 unidades disponibles (stock mínimo fijado: 2 unidades).
- REF-ASU-RYU3-360 (ASUS ROG Ryujin III 360 ARGB LCD): 2 unidades disponibles (stock mínimo fijado: 2 unidades).

13. Reporte 5: Productos Agotados (agotados)

14. *Descripción:* Detecta referencias cuya existencia física en almacén es exactamente cero (0).

15. *Datos obtenidos:* Se registra **1 producto agotado: Gigabyte A520M K V2 Ultra Durable** (Código: MB-GIG-A520M-K), categoría *Placas Madre*, precio Bs. 853.88, existencia 0 unidades.

16. Reporte 6: Resumen por Categoría (por_categoria)

17. *Descripción:* Distribución cuantitativa y económica por categoría de producto.

18. *Datos obtenidos:* **10 categorías activas** con 5 productos cada una:

- *Tarjetas Gráficas:* 5 refs, 35 unidades, valoración Bs. 488,855.80.
- *Monitores:* 5 refs, 24 unidades, valoración Bs. 293,777.12.
- *Procesadores:* 5 refs, 21 unidades, valoración Bs. 125,777.48.
- *Fuentes de Poder:* 5 refs, 20 unidades, valoración Bs. 78,931.60.
- *Almacenamiento:* 5 refs, 31 unidades, valoración Bs. 77,222.28.
- *Placas Madre:* 5 refs, 26 unidades, valoración Bs. 70,516.88.
- *Refrigeración:* 5 refs, 28 unidades, valoración Bs. 47,828.64.
- *Periféricos:* 5 refs, 52 unidades, valoración Bs. 39,283.76.
- *Memorias RAM:* 5 refs, 61 unidades, valoración Bs. 23,288.68.
- *Gabinetes:* 5 refs, 22 unidades, valoración Bs. 16,994.92.

19. Reporte 7: Valor Total del Inventario (valor_total)

20. *Descripción:* Agregación macroeconómica del capital inmovilizado en almacén ($\sum \text{precio} \times \text{cantidad}$).

21. *Datos obtenidos:* Total de referencias activas: **50 productos**; unidades totales físicas: **332 unidades**; valor total consolidado: **Bs. 1,262,477.16**; precio promedio por ítem: Bs. 5,015.30.

22. **Reporte 8: Productos con Mayor Cantidad Disponible (mayor_existencia)**

23. *Descripción:* Artículos con mayor volumen físico de existencias.

24. *Datos obtenidos:* Encabezado por **SteelSeries QcK Heavy XXL Pad Mouse** con **30 unidades**, seguido de **Crucial 8GB DDR4 3200MHz** con **25 unidades**, y **Corsair Vengeance RGB 32GB DDR5** con **14 unidades**.

1.5 Evidencia de Interacciones con Google Antigravity en el Diseño e Implementación del CRUD

Conforme a las directrices de asistencia con IA agéntica, se utilizó **Google Antigravity** para el diseño del modelo relacional, la formulación de validaciones y la construcción de los controladores CRUD. A continuación se presentan las evidencias directas:

Interacción 1.5.1: Modelado de Datos y Validación de Claves

- **Prompt del Desarrollador:**

```
text "Genera un modelo Django en app/models.py para la entidad Producto con más de 10 atributos comerciales para Bolivia (moneda Bs.). Debe tener código único (SKU), categoría indexada, precio positivo, stock físico y mínimo. Además, modela una entidad MovimientoStock para llevar el Kardex físico-valorado de cada entrada, salida y ajuste con usuario y justificación."
```

- **Respuesta y Razonamiento de Antigravity:**

Antigravity propuso el esquema relacional con `PositiveIntegerField` para imposibilitar valores negativos a nivel de base de datos, y un modelo inmutable `MovimientoStock` vinculado mediante `ForeignKey`.

- **Fragmento de Código Generado por Antigravity:**

```
python class MovimientoStock(models.Model): TIPO_CHOICES = [("ENTRADA", "Entrada (+)"), ("SALIDA", "Salida (-)"), ("AJUSTE", "Ajuste de Inventario")] producto = models.ForeignKey(Producto, on_delete=models.CASCADE, related_name="movimientos") tipo = models.CharField(max_length=10, choices=TIPO_CHOICES) cantidad = models.PositiveIntegerField() stock_previo = models.PositiveIntegerField()
```

```
stock_resultante = models.PositiveIntegerField() motivo = models.CharField(max_length=255)
usuario = models.CharField(max_length=100, default="Administrador") fecha =
models.DateTimeField(auto_now_add=True)
```

Interacción 1.5.2: Formularios con Validación Estricta

- **Prompt del Desarrollador:**

```
text "Construye ProductoForm en app/forms.py heredando de forms.ModelForm. Implementa los
métodos clean_codigo, clean_precio y clean_cantidad_existente para asegurar que ningún
precio sea menor a cero y que el código sea único incluso ante modificaciones del
producto."
```

- **Fragmento Generado:** Métodos `clean_*` incorporados exitosamente en `chatbot/app/forms.py` (ver sección 1.3).

Punto 2: Integración con IA local mediante Ollama (40 pts)

2.1 Instalación y configuración de Ollama en Linux

El motor de inteligencia artificial seleccionado opera de forma 100% autónoma, local y soberana mediante el runtime **Ollama** (Ollama, 2024) sobre Debian 12 GNU/Linux.

Comandos de instalación y despliegue del runtime:

```
# 1. Instalación del binario oficial de Ollama en Linux
curl -fsSL https://ollama.com/install.sh | sh

# 2. Verificación de servicio en segundo plano
systemctl is-active ollama
# Salida: active

# 3. Descarga del modelo eficiente Qwen 2.5 (1.5B parámetros cuantizado)
ollama pull qwen2.5:1.5b
```

Salida de descarga obtenida:

```

pulling manifest
pulling 43f7a214e532... 100% ██████████ 986 MB
pulling 62fbfd9ed930... 100% ██████████ 1.1 KB
pulling 56bb8bbc3ed7... 100% ██████████ 96 B
verifying sha256 digest
writing manifest
removing any unused layers
success

```

2.2 Servicio de comunicación Django - Ollama

La interacción se implementó en `chatbot/app/services/ollama_service.py`. El backend actúa como mediador: recibe la consulta del usuario, extrae las métricas reales y el hecho verificado desde el ORM de Django, construye un prompt estructurado y lo remite mediante HTTP POST a `http://localhost:11434/api/generate`.

Fragmento de Código: `chatbot/app/services/ollama_service.py`

```

def consultar_ollama_local(prompt, timeout=80):
    url = obtener_url_ollama()
    modelo = obtener_modelo_activo()
    session = _get_session()

    payload = {
        "model": modelo,
        "prompt": prompt,
        "stream": False,
        "options": {
            "temperature": 0.05,
            "num_predict": 140,
            "num_ctx": 1200,
            "num_thread": 4
        }
    }

    try:
        response = session.post(url, json=payload, timeout=timeout)
        if response.status_code == 200:
            return True, response.json().get("response", "").strip()
            return False, f"Ollama respondió HTTP {response.status_code}"
    except requests.exceptions.ConnectionError:
        return False, "No se pudo conectar con el servicio local de Ollama en http://localh
ost:11434. Verifique que Ollama esté iniciado."
    except requests.exceptions.Timeout:
        return False, f"La consulta a Ollama excedió el tiempo límite ({timeout}s)."

```

2.3 Formulario y vistas del chat en lenguaje natural

La vista del chat soporta tanto peticiones estándar JSON como el estándar de transmisión continua **Server-Sent Events (SSE)** (Mozilla Developer Network [MDN], 2024) para streaming token por token en tiempo real:

Fragmento de Código: Vista `chat` (`chatbot/app/views.py`)

```
@csrf_exempt
@require_http_methods(["POST"])
def chat(request):
    pregunta = request.POST.get("user_input", "").strip()
    if not pregunta:
        return JsonResponse({"error": "La pregunta no puede estar vacía."}, status=400)

    # Restricción de dominio ante preguntas ajenas
    temas_ajenos = ["cocina", "pizza", "receta", "fútbol", "futbol", "mundial", "canción",
"poema", "política", "chiste", "película"]
    if any(t in pregunta.lower() for t in temas_ajenos):
        resp_declinada = "No encontré información suficiente en el inventario para responde
r esa pregunta."
        ChatHistory.objects.create(user_input=pregunta, bot_response=resp_declinada)
        return JsonResponse({"user_input": pregunta, "bot_response": resp_declinada, "cache
d": False})

    # Consulta a Ollama con prompt contextualizado
    prompt_final = construir_prompt_chat(pregunta)
    exito, respuesta = consultar_ollama_local(prompt_final, timeout=80)

    if not exito:
        return JsonResponse({"user_input": pregunta, "bot_response": f"⚠ Error con Ollama:
{respuesta}"})

    ChatHistory.objects.create(user_input=pregunta, bot_response=respuesta)
    return JsonResponse({"user_input": pregunta, "bot_response": respuesta})
```

2.4 Restricción de respuestas y mitigación de alucinaciones

Para garantizar que la IA nunca invente existencias ni precios, el sistema emplea una técnica de **Grounding Determinístico**:

1. La función `resolver_contexto_inteligente(pregunta)` detecta la intención (ej. si el usuario pregunta por el más caro, más barato o agotados) y consulta directamente la base de datos SQLite.

2. Inyecta el resultado como `HECHO OFICIAL VERIFICADO` en el prompt.
3. Se instruye taxativamente al modelo:

"Si la pregunta no está relacionada con los productos o el inventario, responde exclusivamente: 'No encontré información suficiente en el inventario para responder esa pregunta.'"

2.5 Manejo de indisponibilidad y errores de Ollama

Si el proceso de Ollama se encuentra detenido o el puerto 11434 es inaccesible, el backend captura `requests.exceptions.ConnectionError` y retorna un mensaje controlado al usuario:

"No se pudo conectar con el servicio local de Ollama en `http://localhost:11434`. Verifique que Ollama esté iniciado."

2.6 Ejemplos reales de consultas y respuestas obtenidas (Texto plano)

A continuación se presentan transcripciones textuales directas de interacciones reales con el sistema:

- **Caso 1: Consulta del producto más costoso**
 - **Pregunta:** ¿Cuál es el producto más caro del inventario y cuánto cuesta?
 - **Respuesta de Ollama:** El producto más caro de todo el inventario es el ASUS ROG Strix GeForce RTX 4090 24GB OC Edition (Código: GPU-NV-4090-ROG), con un precio de Bs. 26,839.88 y contamos con 8 unidades en existencia.
- **Caso 2: Consulta de artículos sin existencias (Agotados)**
 - **Pregunta:** ¿Qué productos están totalmente agotados actualmente?
 - **Respuesta de Ollama:** Actualmente tenemos 1 producto agotado con 0 unidades disponibles: la placa madre Gigabyte A520M K V2 Ultra Durable (Código: MB-GIG-A520M-K).
- **Caso 3: Consulta global de valoración financiera**

- **Pregunta:** ¿A cuánto asciende el valor total de todo el almacén?
- **Respuesta de Ollama:** El almacén cuenta actualmente con un total consolidado de 332 unidades físicas en existencia, distribuidas en 50 referencias de productos, sumando un valor total económico de Bs. 1,262,477.16.
- **Caso 4: Pregunta fuera de dominio (Prueba de restricción estricta)**
- **Pregunta:** ¿Quién ganó el último mundial de fútbol y cuál es la receta para hacer pizza?
- **Respuesta de Ollama:** No encontré información suficiente en el inventario para responder esa pregunta.

2.7 Evaluación de calidad y limitaciones del modelo local

- **Precisión y Calidad:** Al alimentar al modelo con hechos precalculados mediante el ORM, la tasa de alucinaciones sobre cifras contables se redujo a **0%**. El modelo `qwen2.5:1.5b` comprende perfectamente la terminología comercial y genera redacciones fluidas en español respetando el formato de moneda en Bolivianos (Bs.).
- **Latencia y Rendimiento:** La inferencia en CPU de 4 hilos toma entre **1.2 y 2.4 segundos** por respuesta completa, disminuyendo a milisegundos para preguntas repetidas gracias a la memoria caché.
- **Limitaciones Identificadas:** Modelos compactos de 1.5B parámetros tienen ventanas de contexto más reducidas (~1200 tokens en CPU para mantener baja latencia), lo que impide enviarle simultáneamente las 50 fichas técnicas completas en una sola petición. Por ello, la arquitectura de selección previa y filtrado determinístico implementada en la capa de servicios resulta fundamental.

2.8 Evidencia de Interacciones con Google Antigravity en la Integración con Ollama

La integración del servicio de inferencia local con Ollama fue estructurada y optimizada con la asistencia de **Google Antigravity**:

Interacción 2.8.1: Cliente HTTP Resiliente y Patrón Singleton

- **Prompt del Desarrollador:**

```
text "Genera un servicio en chatbot/app/services/ollama_service.py para consultar la API"
```

de Ollama (<http://localhost:11434/api/generate>) con el modelo `qwen2.5:1.5b`. Aplica un patrón Singleton para `requests.Session` con un pool de conexiones HTTP para evitar agotamiento de sockets. Implementa manejo de errores ante caídas de Ollama (`ConnectionError` y `Timeout`) retornando mensajes claros para el usuario."

- **Respuesta y Razonamiento de Antigravity:**

Antigravity recomendó encapsular la sesión HTTP mediante la función `_get_session()` con `HTTPAdapter(pool_connections=20, pool_maxsize=40)` y timeouts defensivos de 80 segundos.

Interacción 2.8.2: Mitigación Determinística de Alucinaciones

- **Prompt del Desarrollador:**

```
text "Los LLMs pequeños alucinan sumas o precios extremos. Diseña una función resolver_contexto_inteligente(pregunta) que analice la pregunta del usuario, consulte el ORM de Django (más caro, más barato, agotados, total de almacén) e inyecte los datos exactos en el prompt como HECHO OFICIAL VERIFICADO. Si la pregunta es ajena al inventario, fuerza al modelo a responder: 'No encontré información suficiente en el inventario para responder esa pregunta.'"
```

- **Código Generado por Antigravity:**

Implementación en `services/ollama_service.py` con inyección de directivas *System Prompt* y detección semántica determinística (ver sección 2.4).

Punto 3: Calidad, patrones y documentación (30 pts)

3.1 Aplicación de patrones de diseño

El proyecto aplica 4 patrones reconocidos de ingeniería de software basados en el catálogo canónico de patrones de diseño orientado a objetos (Gamma et al., 1994) y patrones de arquitectura empresarial (Fowler, 2002):

1. Patrón Estrategia (*Strategy Pattern*)

Ubicado en `chatbot/app/services/export_service.py`. Conforme a la definición de Gamma et al. (1994), desacopla la familia de algoritmos de exportación multiformato mediante una clase base abstracta:

```

from abc import ABC, abstractmethod

class BaseExportStrategy(ABC):
    @abstractmethod
    def exportar(self, formato="csv", **kwargs) -> HttpResponse:
        pass

class ProductCatalogExportStrategy(BaseExportStrategy):
    def exportar(self, formato="csv", **kwargs) -> HttpResponse:
        if formato == "csv":
            # Genera CSV compatible con Microsoft Excel (UTF-8 con BOM)
            ...
        elif formato in ("html", "pdf"):
            # Genera Hoja Oficial de Inventario con firmas y diseño ejecutivo
            ...

```

2. Patrón Singleton (Pool de Conexiones Persistentes HTTP)

Ubicado en `chatbot/app/services/ollama_service.py`. Implementa una variante del patrón creacional Singleton (Gamma et al., 1994) para garantizar una única instancia global de `requests.Session` con keep-alive y reintentos adaptativos, protegiendo los descriptors de sockets del sistema operativo:

```

_session = None

def _get_session():
    global _session
    if _session is None:
        _session = requests.Session()
        adapter = HTTPAdapter(pool_connections=20, pool_maxsize=40, max_retries=Retry(total=2, backoff_factor=0.2))
        _session.mount('http://', adapter)
    return _session

```

3. Patrón Capa de Servicios (Service Layer)

Siguiendo las directrices arquitectónicas de Fowler (2002), se desacopla la capa de presentación (`views.py`) distribuyendo la responsabilidad en módulos de dominio especializados: `inventory_service.py` (reglas de negocio y reportes), `ollama_service.py` (integración con IA local) y `export_service.py` (serialización multiformato).

4. Patrón Registro Inmutable / Audit Ledger (Kardex Físico-Valorado)

Encapsulado en el modelo `MovimientoStock`, asegurando que ninguna modificación física de almacén ocurra sin una entrada auditable con sello de tiempo, usuario y justificación.

3.2 Batería de pruebas unitarias automatizadas

Se implementaron **17 pruebas unitarias** en `chatbot/app/tests.py` que evalúan exhaustivamente las funcionalidades críticas del sistema (código único, stock no negativo, reportes, Kardex, streaming SSE y exportaciones).

Comando de ejecución:

```
python chatbot/manage.py test app -v 2
```

Salida completa de terminal:

```

Found 17 test(s).
Creating test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...
Operations to perform:
  Synchronize unmigrated apps: messages, staticfiles
  Apply all migrations: admin, app, auth, contenttypes, sessions
Synchronizing apps without migrations:
  Creating tables...
  Running deferred SQL...
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying app.0001_initial... OK
  Applying app.0002_alter_chathistory_options_and_more... OK
  Applying app.0003_producto... OK
  Applying app.0004_actualizar_producto... OK
  Applying app.0005_alter_producto_cantidad_existente_and_more... OK
  Applying app.0006_producto_fecha_actualizacion_and_more... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
System check identified no issues (0 silenced).
test_chat_stream_sse_endpoint (app.tests.InventarioCRDTests.test_chat_stream_sse_endpoint)
Verifica que el endpoint de streaming SSE retorne un stream con Content-Type text/event-stream. ... ok
test_exactitud_agotados_y_extremos_inventario (app.tests.InventarioCRDTests.test_exactitud_agotados_y_extremos_inventario)
Verifica que el resolver inteligente identifique exactamente los productos agotados y extremos sin alucinaciones. ... ok
test_exportacion_catalogo_csv (app.tests.InventarioCRDTests.test_exportacion_catalogo_csv)
Verifica la exportación en tiempo real del catálogo a formato CSV. ... ok
test_exportacion_catalogo_pdf_hoja_oficial (app.tests.InventarioCRDTests.test_exportacion_catalogo_pdf_hoja_oficial)
Verifica la generación de la Hoja Oficial de Inventario Valorizado imprimible en PDF. ... ok
test_importacion_catalogo_csv_valido (app.tests.InventarioCRDTests.test_importacion_catalogo_csv_valido)
Verifica la importación masiva de productos desde archivo CSV. ... ok
test_kardex_registro_movimientos_y_consulta_api (app.tests.InventarioCRDTests.test_kardex_r

```

```

registro_movimientos_y_consulta_api)
Verifica el registro atómico de movimientos en el Kardex y su consulta vía API. ... ok
test_optimizacion_cache_chat (app.tests.InventarioCRDTests.test_optimizacion_cache_chat)
Verifica que las consultas repetidas respondan desde la caché en milisegundos. ... ok
test_pagina_principal_carga_correctamente (app.tests.InventarioCRDTests.test_pagina_princip
al_carga_correctamente)
Verifica que la interfaz cargue con HTTP 200 y contenga la información de red. ... ok
test_paginacion_catalogo_productos (app.tests.InventarioCRDTests.test_paginacion_catalogo_p
roductos)
Verifica la paginación interactiva del catálogo sin recarga de página. ... ok
test_rf01_registro_producto_exitoso (app.tests.InventarioCRDTests.test_rf01_registro_produc
to_exitoso)
RF-01: Registro de un nuevo producto con campos obligatorios y código único. ... ok
test_rf01_registro_producto_falla_por_codigo_duplicado_o_precio_negativo (app.tests.Inventa
rioCRDTests.test_rf01_registro_producto_falla_por_codigo_duplicado_o_precio_negativo)
RF-01: Validación de que no se permitan códigos duplicados ni precios negativos. ... ok
test_rf02_consulta_y_busqueda_productos (app.tests.InventarioCRDTests.test_rf02_consulta_y_
busqueda_productos)
RF-02: Consulta de productos y búsqueda por código, nombre o categoría. ... ok
test_rf03_actualizacion_producto (app.tests.InventarioCRDTests.test_rf03_actualizacion_produ
cto)
RF-03: Modificar información de un producto existente. ... ok
test_rf04_eliminacion_logica_y_cambio_estado (app.tests.InventarioCRDTests.test_rf04_elimina
cion_logica_y_cambio_estado)
RF-04: Eliminación lógica cambiando estado a inactivo para conservar historial. ... ok
test_rf05_control_existencia_aumentar_y_disminuir (app.tests.InventarioCRDTests.test_rf05_c
ontrol_existencia_aumentar_y_disminuir)
RF-05: Aumentar y disminuir existencias impidiendo valores negativos. ... ok
test_rf06_menu_reportes_predefinidos_8_opciones (app.tests.InventarioCRDTests.test_rf06_men
u_reportes_predefinidos_8_opciones)
RF-06: Verificación de los 8 reportes predefinidos requeridos. ... ok
test_rf07_rf08_rf09_chat_con_ollama (app.tests.InventarioCRDTests.test_rf07_rf08_rf09_chat_
con_ollama)
RF-07, RF-08, RF-09: Chat interactivo con contexto oficial de inventario y restricción. ...
ok

-----
Ran 17 tests in 0.096s

OK
Destroying test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shar
ed')...

```

3.3 Documentación y arquitectura técnica

El repositorio contiene la documentación estructurada del sistema:

- * [README.md](#): Guía de despliegue nativo y contenedores Docker.
- * [CAMBIOS.md](#): Bitácora de evolución desde el prototipo base hasta la Actividad 5.
- * [DOCUMENTACION.md](#): Especificación detallada de decisiones técnicas, patrones y arquitectura en

capas.

* [ANTIGRAVITY.md](#): Bitácora detallada de sesiones, prompts, respuestas y código generado con el asistente IA Google Antigravity.

* [OPENCODE.md](#): Registro complementario de conformidad para asistentes agénticos de codificación.

3.4 Gestión de dependencias y variables de entorno

- [requirements.txt](#): Dependencias congeladas del entorno virtual (`Django==5.1.6` , `requests==2.32.3` , `cachetools==7.2.0` , `python-dotenv==1.2.3` , `ollama==0.4.7`).
- [.env.example](#): Plantilla con las variables obligatorias (`OLLAMA_URL` , `OLLAMA_MODEL=qwen2.5:1.5b` , `SECRET_KEY` , `DEBUG` , `ALLOWED_HOSTS`).

3.5 Documentación del uso de Google Antigravity: sesiones realizadas, prompts utilizados, respuestas obtenidas y su impacto en el desarrollo

En concordancia con el subpunto 3.5 de la consigna, la refactorización hacia patrones de diseño y la construcción de la batería de pruebas automatizadas fueron ejecutadas con el soporte interactivo de **Google Antigravity**:

Sesión de Patrones de Diseño (Refactorización):

- **Prompt del Desarrollador:**

```
text "Aplica el Patrón Estrategia (Strategy Pattern) en services/export_service.py para
desacoplar las exportaciones multiformato. La clase base BaseExportStrategy debe definir
exportar(). Implementa ProductCatalogExportStrategy para exportar en CSV compatible con
Excel (UTF-8 con BOM) y en Hoja Oficial imprimible en HTML/PDF con casillas de firma
formal de auditoría y almacén."
```

- **Respuesta y Razonamiento de Antigravity:**

Antigravity diseñó una jerarquía polimórfica que permite incorporar nuevos formatos sin alterar `views.py` (principio Open/Closed), generando además la plantilla ejecutiva de la Hoja Oficial.

Sesión de Pruebas Unitarias Automatizadas:

- **Prompt del Desarrollador:**

```
text "Genera 17 pruebas unitarias con Django TestCase en chatbot/app/tests.py que validen el 100% de los requerimientos: unicidad de SKU, precios no negativos, paginación, los 8 reportes corporativos predefinidos, cálculo del Kardex físico-valorado, streaming SSE y exportaciones. Asegura que operen sobre SQLite en memoria y se ejecuten en menos de un segundo."
```

- **Impacto Obtenido:**

Las 17 pruebas fueron formuladas y aprobadas en **0.11 segundos**, garantizando que el sistema sea inmune a regresiones operativas.

Uso de Google Antigravity (Asistente de Codificación IA)

De acuerdo con las instrucciones de la cátedra que autorizan la selección libre de herramientas de inteligencia artificial de desarrollo agéntico, se utilizó **Google Antigravity** (Google DeepMind, 2024) a lo largo de todo el ciclo de desarrollo del proyecto, contrastado con asistentes abiertos convencionales como OpenCode (OpenCode Project, 2024).

En la Tabla 2 se sintetizan las sesiones de trabajo registradas con el asistente agéntico:

Tabla 2

Bitácora Consolidada de Sesiones de Trabajo Asistidas por Google Antigravity

N° Sesión	Fase del Proyecto	Herramienta / Modelo	Prompts Principales	Salidas Generadas
S1	Modelado CRUD y Kardex	Google Antigravity	Definición de campos de Producto y MovimientoStock	app/models.py , app/forms.py , app/admin.py
S2	Capa de Servicios y Reportes	Google Antigravity	Desacoplamiento de vistas y cálculo de 8 reportes	app/services/inventory_service.py
S3	Integración IA Ollama	Google Antigravity	Singleton HTTP, Grounding determinístico y SSE	app/services/ollama_service.py , app/views.py
S4	Patrones de Diseño	Google Antigravity	Patrón Strategy para exportación y Ledger inmutable	app/services/export_service.py
S5	Pruebas Automatizadas	Google Antigravity	Batería de 17 tests unitarios en base de datos en memoria	app/tests.py
S6	Auditoría y Documentación	Google Antigravity	Cumplimiento de rúbrica, documentación y empaquetado	DOCUMENTACION.md , ANTIGRAVITY.md , informe.md

Nota. Registro cronológico de interacciones, requerimientos y artefactos técnicos generados durante el ciclo de vida del software asistido por IA agéntica (Google DeepMind, 2024).

Impacto de Antigravity en la Calidad del Software

1. **Erradicación de Alucinaciones:** Permitió diseñar la inyección contextual desde el ORM de Django antes de invocar a Ollama, eliminando errores aritméticos propios de modelos pequeños (1.5B).
2. **Arquitectura Limpia:** Fomentó la separación estricta de responsabilidades (Service Layer, Strategy), evitando el antipatrón de vistas recargadas.

3. **Resiliencia Operativa:** Estableció manejo defensivo de excepciones (`requests.exceptions.ConnectionError`), asegurando que si el daemon de Ollama no está activo, el sistema presente un mensaje comprensible en lugar de una pantalla de error 500.
-

Reflexión técnica: Dificultades encontradas, soluciones y aprendizajes

1. Dificultad: Mitigación de Alucinaciones en Cálculos Numéricos con LLMs

- **El Problema:** Los modelos de lenguaje pequeños (como los de 1.5B parámetros) presentan dificultades intrínsecas para sumar grandes listas o deducir el valor mínimo/máximo de una colección de 50 elementos enviada en texto crudo, tendiendo a inventar cifras o confundir monedas.
- **Cómo se resolvió:** Se implementó una arquitectura híbrida de *Grounding Determinístico*. Antes de despachar el prompt a Ollama, el backend en Python consulta la base de datos con el ORM de Django, calcula las cifras exactas y las inyecta en el prompt etiquetadas como *Hecho Oficial Verificado*. De esta forma, el LLM se encarga únicamente de formular la redacción en lenguaje natural, eliminando el error matemático.
- **Qué se aprendió:** Los modelos de IA no deben emplearse como motores de cálculo numérico, sino como sintetizadores y redactores de información estructurada previamente validada por el backend.

2. Dificultad: Concurrencia y Bloqueos en SQLite

- **El Problema:** Durante pruebas concurrentes de streaming en el chat y registro de movimientos de Kardex, SQLite arrojaba intermitentemente advertencias de bloqueo (`database is locked`) debido al modo tradicional de journal.
- **Cómo se resolvió:** Se activó el modo **WAL (Write-Ahead Logging)** en SQLite con `PRAGMA journal_mode = WAL;` y `PRAGMA synchronous = NORMAL;`, lo que permite lecturas concurrentes simultáneas sin bloquear las transacciones de escritura.
- **Qué se aprendió:** Comprender el mecanismo de almacenamiento de los motores de bases de datos relacionales es indispensable para evitar cuellos de botella en aplicaciones web interactivas.

3. Dificultad: Latencia de Inferencia en Equipos sin GPU Dedicada

- **El Problema:** La respuesta completa de un LLM en CPU estándar generaba pausas perceptibles de 2 a 3 segundos, lo cual degradaba la experiencia de usuario.
 - **Cómo se resolvió:** Se implementaron dos soluciones complementarias: una capa de caché en memoria (`TTLCache`) para devolver consultas idénticas en milisegundos, y un endpoint de **Server-Sent Events (SSE)** con `StreamingHttpResponse` para transmitir los tokens de respuesta progresivamente a la interfaz.
 - **Qué se aprendió:** Las técnicas de streaming y caché son esenciales para lograr interfaces fluidas y receptivas en aplicaciones impulsadas por modelos de inteligencia artificial locales.
-

Referencias

Django Software Foundation. (2024). *Django: The web framework for perfectionists with deadlines* (Versión 5.1.6) [Software de computadora]. <https://docs.djangoproject.com/>

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley Professional.

Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Google DeepMind. (2024). *Google Antigravity: Advanced agentic AI coding assistant* [Software de computadora]. <https://deepmind.google/technologies/antigravity>

Mozilla Developer Network. (2024). *Server-sent events*. MDN Web Docs. https://developer.mozilla.org/es/docs/Web/API/Server-sent_events

Ollama. (2024). *Ollama: Get up and running with large language models locally* (Versión 0.4.7) [Software de computadora]. <https://ollama.com/>

OpenCode Project. (2024). *OpenCode AI: Open-source coding agent* [Software de computadora]. <https://opencode.ai/>